

Software Engineering Lab

Date: 20-08-2026

Name: Shreyashree S Rao

SRN: PES1UG24AM267

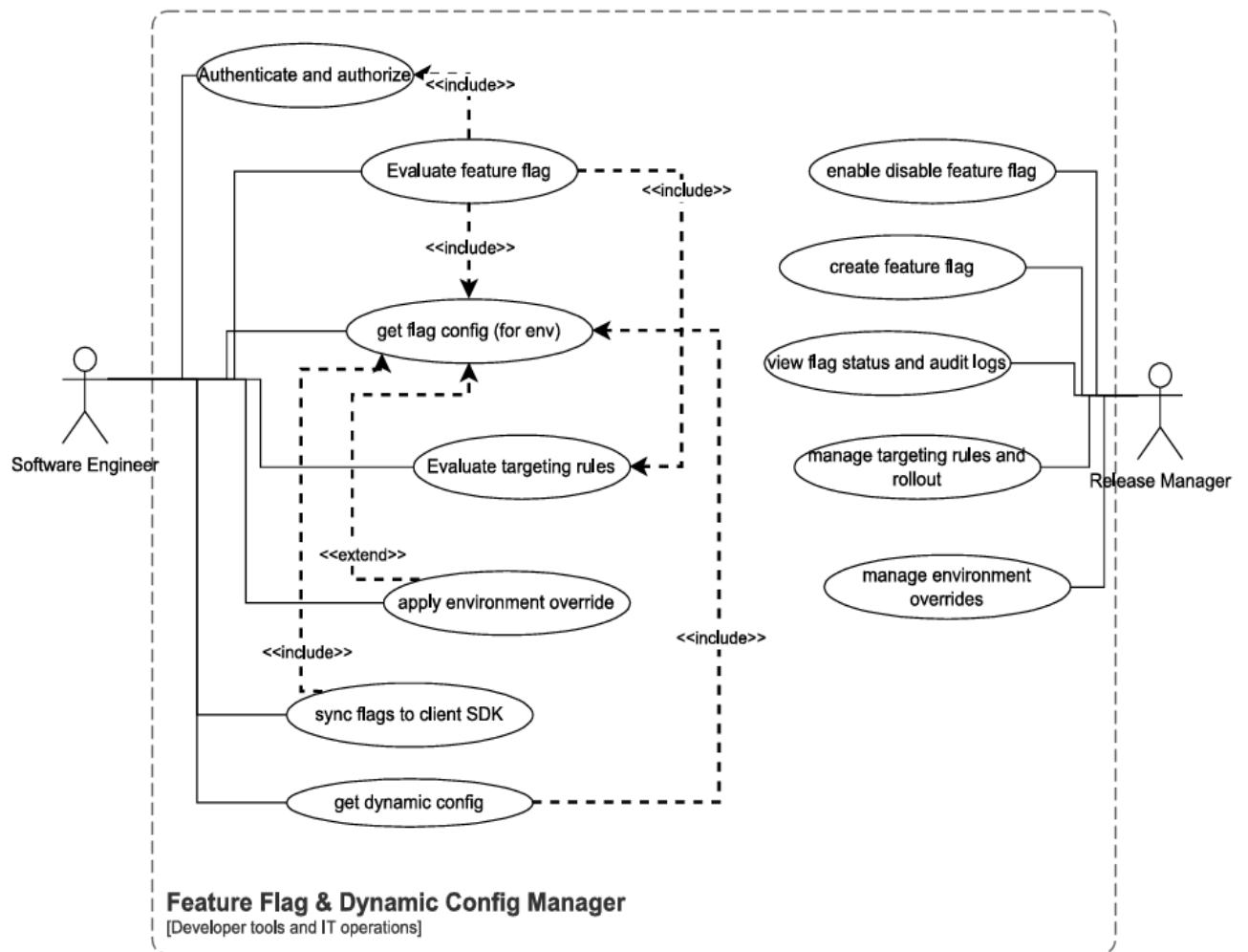
Section: E

Requirements table:

ID	type	description	priority	criteria for acceptance	rationale
FR-001	functional	The system shall evaluate user targeting rules, including User ID hashing and beta-cohort membership, and return the appropriate boolean feature-flag state.	high	Pass: a user belonging to a configured 10% rollout cohort receives the enabled flag Fail: a user outside the configured cohort does not receive the enabled flag	enables controlled and predictable feature rollouts to specific user groups while reducing deployment risk
FR-002	functional	The system shall support percentage-based feature-flag rollouts by deterministically assigning users to rollout cohorts.	high	Pass: when a feature is configured for a 25% rollout, approximately 25% of eligible users are assigned the enabled state consistently across evaluations Fail: the same user is randomly assigned different states for the same configuration	provides gradual feature deployment while ensuring stable user experiences
FR-003	functional	The system shall support environment-specific configuration overrides for feature flags and dynamic configuration values.	high	Pass: a flag configured differently for development, staging, and production returns the value corresponding to the requesting environment Fail: a production request receives a development or staging configuration	allows teams to test and release features independently across environments without modifying application code

FR-004	functional	The system shall provide real-time synchronization of feature-flag states and configuration changes to connected client SDKs.	medium	Pass: a configuration change made by an authorized user is propagated to connected clients without requiring an application restart Fail: connected clients continue using the previous configuration after the synchronization period	ensures clients receive current configuration promptly and avoids stale feature-flag states
FR-005	functional	The system shall allow authorized users to create, update, enable, disable, and manage feature flags and their targeting configurations.	medium	Pass: an authorized Release Manager can modify a feature flag and its rollout rules successfully Fail: an unauthorized user is prevented from modifying the configuration	Provides controlled management of feature flags while preventing unauthorized configuration changes.
NFR-001	performance and security	The feature-flag evaluation API shall execute within 15 ms under normal operating conditions and maintain the required performance under simulated peak load.	high	Pass: benchmarking confirms that flag evaluation meets the 15 ms latency target under the defined test load Fail: the API consistently exceeds the latency threshold	Feature-flag evaluation may occur on critical application paths, so excessive latency could directly affect application performance.
NFR-002	security and reliability	The system shall enforce authenticated and authorized access to feature-flag and configuration management operations and maintain consistent flag states during client synchronization.	high	Pass: only authenticated users with appropriate permissions can modify configurations, and connected clients receive the resulting valid configuration Fail: an unauthorized modification is accepted or clients receive an invalid/inconsistent configuration	Protects application behaviour from unauthorized configuration changes and reduces the risk of inconsistent system states.

UML Use-Case Diagram:



Shreyashree S Rao
PES1UG24AM267

Use-case flow specification:

Use case: evaluate feature flag

Primary actor: software engineer and client SDK

Brief description: to check whether a particular feature should be enabled for a user

Preconditions:

1. the feature flag is already created in the system
2. the feature flag has its targeting rules and environment settings configured
3. the user and environment details are available
4. the client has a valid API key/token to request the flag

Postconditions:

1. the system returns either enabled or disabled for the feature flag
2. the result that is returned is based on the configured targeting rules and environment
3. no changes are made to the feature flag configuration

Main Success Scenario:

1. the client sends a request to check a feature flag for a particular user
2. the system checks the request and identifies the requested feature flag
3. the system gets the feature flag settings for the given environment
4. the system checks the targeting rules, such as user id, beta user status or percentage rollout
5. the system determines whether the user matches the given rules
6. the system returns the feature flag status as enabled
7. the client uses this result to decide whether to show or use the feature

Alternate flow – user does not match the rules:

1. the system checks the targeting rules and finds that the user does not match them
2. the system sets the feature flag status to disabled for that user
3. the system returns disabled to the client
4. the client does not enable the feature for that user
5. use case ends